
RSN MATCHING SYSTEM SPECIFICATION (v1)

1. PURPOSE

The RSN matching system exists to create the best possible set of conversations across an event.

It must:

- match people based on intent and relevance
- avoid repeat conversations
- balance fairness across all participants
- support premium preferences without breaking the system
- learn from real outcomes over time
- adapt dynamically to joins, leaves, and failures

This is not random matching. It is a constrained, learning system.

2. DEFINITIONS

- User: a participant in an event
- Session: one round of meetings
- Event: a set of sessions (default 5)
- Pod: a recurring community or group
- Match: a pair or group assigned to meet
- Pair: two users considered for matching
- Three person room: group of three users when needed

3. MATCHING PRINCIPLES

- Plan globally, not round by round
- Never repeat the same pair in the same event unless impossible
- Prioritize strong intent and useful conversations
- Balance experience across all users
- Prefer new connections, but allow valuable repeats when justified
- Learn from real feedback, not just profiles
- Adapt without breaking active sessions

4. REQUIRED DATA MODEL

User Profile

user_id
role
industry
stage
goals
intent
preferences
premium_flag
requested_users
blocked_users

Event Participation

event_id
user_id
join_time
leave_time
status (joined, no show, left early)

Meeting History

user_A
user_B
event_id
pod_id
timestamp
rating_A
rating_B
meet_again_A
meet_again_B

Pair Relationship

user_A
user_B
total_meetings
last_meeting
average_rating
mutual_meet_again
avoid_flag

Session Assignment

event_id
session_number
user_A
user_B
match_score
match_reason
fallback_flag
repeat_flag

5. PRE EVENT PLANNING

Before the event starts:

load all participants
remove no shows
load meeting history and relationships
calculate all possible pair scores
check feasibility of unique matches

Generate the full session plan upfront.

Do not match session by session.

6. MATCHING RULES

Hard rules:

no user appears twice in same session
no pair repeats inside same event unless impossible
blocked users never match

Priority order:

safety blocks
user blocks
no repeat inside event
premium requests
host curated matches
intent fit
pod history avoidance
platform history avoidance
fairness
fallback

7. PREMIUM MATCHING

Premium users can:

- request specific people
- receive higher probability of matching

Rules:

- premium cannot override safety or blocks
- premium cannot force a match
- mutual premium requests have highest priority
- single requests have strong priority
- premium cannot dominate all matches
- requested matches occur at most once per event

8. FEEDBACK AND LEARNING

After each session:

- collect usefulness rating
- collect meet again preference

Interpretation:

- both users positive → strong affinity
- both users negative → avoid future matching
- one sided negative → deprioritize

Learning:

- increase probability of successful match patterns
- decrease probability of failed patterns

The system must learn from behavior, not just profiles.

9. DYNAMIC REPAIR

The system must adapt during the event.

Rules:

never change a live session
only update future sessions

If user joins late:

add between sessions
recalculate future sessions only

If user leaves:

remove from future sessions
repair only affected matches

If disconnection:

under 60 seconds keep match
60 to 180 seconds reconnect state
over 180 seconds treat as left

Repair priority:

preserve completed sessions
preserve current session
fix only broken future matches
avoid full rebuild unless necessary

10. EDGE CASES

Odd number of users:

create three person rooms
rotate fairly
avoid repeating same users in these rooms

No shows:

exclude before session one

Late joiners:

only enter between sessions

Not enough valid matches:

apply fallback logic

Fallback order:

- allow platform repeats
- allow pod repeats
- allow recent repeats
- only then repeat inside same event

Never match blocked users

11. ADMIN CONTROLS

Host must control:

- number of sessions
- session duration
- matching scope (event, pod, platform)
- allow three person rooms
- allow repeat matches
- priority of intent matching
- priority of new connections

Default:

- 5 sessions
- pod level memory
- no repeats inside event
- allow fallback
- prioritize intent

12. USER STATES

User can be:

- waiting
- matched
- in session
- late joiner
- disconnected
- stranded
- in three person room
- blocked

System must clearly handle each state.

13. LOGGING AND DEBUGGING

For each match store:

match_score
reason for match
whether premium influenced
whether fallback used
whether users met before

This is required for debugging and improvement.

14. ACCEPTANCE CRITERIA

The system is complete when:

a full event plan is generated before start
no duplicate matches occur inside event unless unavoidable
each user gets one match per session
fairness is maintained across participants
premium requests are respected but not dominant
feedback is collected after each session
future matching improves based on feedback
system adapts to joins and leaves without breaking
no blocked users are ever matched
edge cases are handled without failure

Final principle

Plan globally
Execute session by session
Repair only the future
Learn continuously
Balance the experience for everyone
